
Fundamentos C#

1. Contato inicial com c#

```
using System;

namespace Console01
{
    0 referências
    internal class Program
    {
        0 referências
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

Namespace: É uma área de escopo que contém um conjunto de classes relacionadas. É uma prática comum organizar o código em namespaces para evitar conflitos de nome e manter a estrutura do código bem organizada.

Classe Program: A classe principal de um aplicativo de console em C# é chamada de "Program". Esta classe contém o método "Main".

Método Main: É por onde a execução do programa começa. O parâmetro "args" do tipo "string[]" representa os argumentos da linha de comando passados para o programa quando ele é iniciado.

<Escrever na tela

- WriteLine: Pula linha no final.
- Write: Não pula linha

```
Console.Write("Hello ");
Console.WriteLine("World!");
```

<Declarando variável e constante

```
int idade = 20;
const string nome = "Wesley";
```

2. Tipos

Built-in types

1. Inteiros

byte	1 byte	0 ... 255	0	byte varByte = 0;
sbyte	1 byte	-128 ..127	0	sbyte varSByte = 0;
short	2 bytes	-32,768 ..32,767	0	short varShort = 0;
ushort	2 bytes	0 ..65,535	0	ushort varUShort = 0;
int	4 bytes	-2,147,483,648 ..2,147,483,647	0	int varInt = 0;
uint	4 bytes	0 ..4,294,967,295	0	uint varUInt = 0;
long	8 bytes	-9,223,372,036,854,775,808..9,223,372,036,854,775,807	0	long varLong = 0L;
ulong	8 bytes	0 ..18,446,744,073,709,551,615	0	ulong varULong = 0;

2. Reais

float	4 bytes	-3.402823e38 ..3.402823e38	0	float varFloat = 0f;
double	8 bytes	-1.79769313486232e308 ..1.79769313486232e308	0	double varDouble = 0.0;
decimal	16 bytes	-79228162514264337593543950335..79228162514264337593543950335	0	decimal varDecimal = 0.0m;

Para float e decimal é necessário usar sufixos.

```
float altura = 1.75f;  
decimal salario = 4000.1m;
```

3. Boolean

- bool: representa valores booleanos.

4. Caracter

- char: representa um único caractere.
- string: representa uma sequência de caracteres.

Nullable Types

Os nullable types são tipos que podem armazenar valores do valor especial `null`. Para criar um tipo nullable, você deve adicionar um (?) após o tipo.

```
int? idade = null;
```

Conversão de tipos

1. Conversão implícita: Ocorre quando não há perda de dados ao converter de um tipo para outro. O compilador é capaz de realizar essa conversão automaticamente.

```
int a = 5;  
float b = a;
```

2. Conversão explícita: ocorre quando há uma possível perda de dados. Para realizar, você precisa usar um operador de cast.

```
float peso = 72.999f;  
int pesoInt = (int)peso;
```

3. Conversão por métodos auxiliares: A classe estática "Convert" fornece uma série de métodos estáticos para converter valores entre diferentes tipos de dados.

```
float peso = 72.999f;  
int pesoInt = Convert.ToInt16(peso);
```

4. TryParse: TryParse é uma maneira segura de converter um string em número. Ele é especialmente útil quando você está lidando com entrada de usuário.

```
string numero = "2000";  
int a;  
int.TryParse(numero, out a);
```

3. Leitura de dados

1. **Console.ReadLine():** Lê uma linha de entrada e a retorna como uma string.

```
string name = Console.ReadLine();
```

2. **Como ler valores numéricos:** Usa-se o TryParse para converter o retorno.

```
int idade;  
int.TryParse(Console.ReadLine(), out idade);
```

4. Mathematic

Biblioteca math.

```
decimal moeda = 3400.56m;  
//Abs: pega o valor absoluto  
Console.WriteLine(Math.Abs(-100));  
//Round: arredonda  
Console.WriteLine(Math.Round(moeda));  
//Ceiling: pega o teto  
Console.WriteLine(Math.Ceiling(moeda));  
//Floor: pega o chao  
Console.WriteLine(Math.Floor(moeda));  
//Truncate: retorna a parte inteira  
Console.WriteLine(Math.Truncate(moeda));  
  
//Pow: exponenciacao  
Console.WriteLine(Math.Pow(10,2));  
//Sqrt: Raiz quadrada  
Console.WriteLine(Math.Sqrt(36));  
  
//Constantes matematicas  
Console.WriteLine(Math.PI);  
Console.WriteLine(Math.E);
```

Classe Random

Utilizada para gerar números aleatórios. É necessário criar um gerador do tipo Random e usar next(). Neste exemplo, será gerado um número inteiro entre 1 e 9.

```
Random gerador = new Random();
Console.WriteLine(gerador.Next(1, 10));
```

Para gerar números com ponto flutuante usa-se o NextDouble(). Gera entre 0 e 1.

```
Random gerador = new Random();
Console.WriteLine(gerador.NextDouble());
```

5. Trabalhando com strings

1. Interpolação/concatenação

```
string nome = "Wesley";
int idade = 18;
float altura = 1.74f;
Console.WriteLine($"Olá meu nome é {nome}, tenho {idade} anos" +
    $" e tenho {altura} de altura");
```

2. Formatação numéricas interpoladas

Console.WriteLine(\$"Alinhamento a dir: {idade, 6}.");	Alinhamento a dir: 18.
Console.WriteLine(\$"Alinhamento a esq: {idade,- 6}.");	Alinhamento a esq: 18
Console.WriteLine(\$"Formatação de moeda: {salario:c}");	Formatação de moeda: \$3,500.90
Console.WriteLine(\$"Formatação num float: {altura:F2}");	Formatação num float: 1.74
Console.WriteLine(\$"Formatação com sep: {habitantes:N}");	Formatação com sep: 10,000,000.00
Console.WriteLine(\$"Formatação num int: {idade:D3}");	Formatação num int: 018

3. Comparações

string.Contains("text") -> retorna boolean
string.Equals("text") -> retorna boolean
string.StartsWith("text") -> retorna boolean
string.EndsWith("text") -> retorna boolean

4. Índices

string.IndexOf("txt") -> retorna o índice da primeira ocorrência (-1 se não tiver)
string.LastIndexOf("txt") -> retorna o índice da última ocorrência (-1 se não tiver)

5. Mais métodos

<code>string.Insert(i, "text")</code>	-> insere 'text' no índice 'i'
<code>string.Remove(startindex, count)</code>	-> remove
<code>string.ToLower()</code>	-> deixa o texto minúsculo
<code>string.ToUpper()</code>	-> Deixa o texto maiúsculo
<code>string.Replace("text", "newtext")</code>	-> Substitui todas as ocorrências
<code>string.Trim()</code>	-> remove espaços do começo e do final
<code>string.Length()</code>	-> tamanho da string
<code>string.Split(" ")</code>	-> separa as palavras com base no arg passado

6. StringBuilder

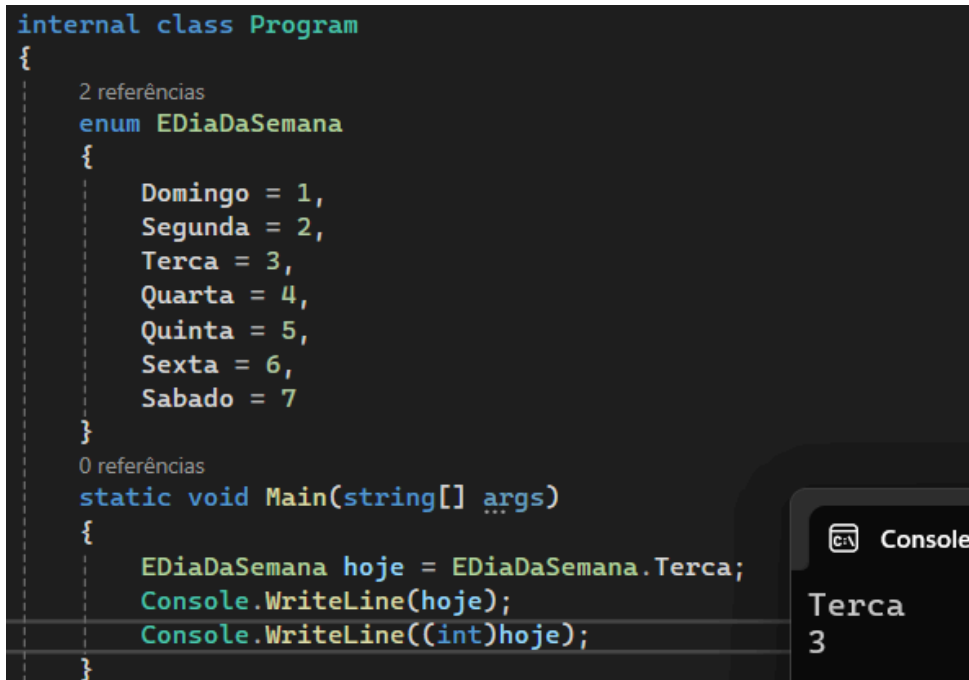
Está no namespace `System.Text` e é usado para manipular eficientemente strings mutáveis. A principal vantagem do `StringBuilder` em comparação com `string` é que o `StringBuilder` evita a criação desnecessária de objetos de string intermediários.

```
StringBuilder ss = new StringBuilder();  
ss.AppendLine("Integrantes:");  
ss.Append(" Julio,");  
ss.Append(" Jessica.");  
ss.ToString();
```

6. Enum

Enum, abreviação de "enumeration" é um tipo de dados em C# que nos permite definir um conjunto de constantes nomeadas e numeradas.

```
internal class Program
{
    2 referências
    enum EDiaDaSemana
    {
        Domingo = 1,
        Segunda = 2,
        Terca = 3,
        Quarta = 4,
        Quinta = 5,
        Sexta = 6,
        Sabado = 7
    }
    0 referências
    static void Main(string[] args)
    {
        EDiaDaSemana hoje = EDiaDaSemana.Terca;
        Console.WriteLine(hoje);
        Console.WriteLine((int)hoje);
    }
}
```



7. Switch case

```
EDayOfWeek Current = EDayOfWeek.Sunday;
switch((int)Current)
{
    case(1): Console.WriteLine("Domingo"); break;
    case(2): Console.WriteLine("Segunda"); break;
    case(3): Console.WriteLine("Terça"); break;
    case(4): Console.WriteLine("Quarta"); break;
    case(5): Console.WriteLine("Quinta"); break;
    case(6): Console.WriteLine("Sexta"); break;
    case(7): Console.WriteLine("Sábado"); break;
    default: Console.WriteLine("Indefinido"); break;
}
```

8. If ternário

O operador ternário é uma forma compacta de expressar uma condição simples.

1. Sintaxe: var tipo = (condição)? "Caso V" : "Caso F";

```
if((idade < 18)? false : true)
{
    Console.WriteLine("Maior de idade");
}
else
{
    Console.WriteLine("Menor de idade");
}
```

9. Estruturas de repetição

<For

```
for(int i=0; i < 10; i++){
    Console.WriteLine(i);
}
```

<Foreach

```
string text = "Ola meu nome é wesley";
string[] palavras = text.Split(" ");

foreach(string palavra in palavras){
    Console.WriteLine(palavra);
}
```

<While

```
int i = 0;
while(i < 10){
    Console.WriteLine(i);
    i++;
}
```

10. Tipos anônimos

Tipos anônimos permitem a criação de objetos sem a necessidade de definir explicitamente uma classe. Eles são particularmente úteis para armazenar temporariamente um conjunto de valores relacionados de maneira simples.

```
var person = new
{
    Name = "Joao",
    Age = 19
};
```

11. Tupla

É uma estrutura de dados que permite combinar múltiplos valores em um único objeto, sem a necessidade de criar uma classe.

1. Declarando

```
//Primeira maneira
(int Id, String Name, float Altura) person1 = (18, "Wesley", 1.73F);
//Segunda maneira
ValueTuple<int, string, float> person2 = (20, "Lucas", 1.77F);
```

2. Acessando

```
Console.WriteLine($"Idade: {person1.Idade}");
Console.WriteLine($"Nome: {person1.Name}");
Console.WriteLine($"Idade: {person2.Item1}");
Console.WriteLine($"Nome: {person2.Item2}");
```

3. Desconstruindo

```
(int Idade, String Name, float Altura) person1 = (18, "Wesley", 1.73F);

var(idade, name, altura) = person1;

Console.WriteLine($"Idade:{idade}");
Console.WriteLine($"Name:{name}");
Console.WriteLine($"Idade:{altura}");
```